

DATA STRUCTURES – IMPORTANT QUESTIONS

Topic-wise question bank for examination preparation

Tip: Prepare definitions and short questions first, then algorithms, examples, diagrams and complexity for long-answer questions.

1. Introduction to Data Structures

1. Define data structure. Explain its need and applications.
2. Differentiate primitive and non-primitive data structures.
3. Differentiate linear and non-linear data structures with examples.
4. What is an Abstract Data Type (ADT)? Give examples.
5. Explain time complexity and space complexity.
6. Explain Big-O notation with common complexity classes.

2. Arrays

1. Define an array. Explain its characteristics and memory representation.
2. Explain array insertion and deletion with an example.
3. Explain traversal and searching in an array.
4. Compare arrays and linked lists.
5. Write an algorithm for insertion/deletion in an array.
6. Discuss the advantages and disadvantages of arrays.

3. Linked Lists

1. Define linked list. Explain the structure of a node.
2. Explain singly linked list operations: insertion, deletion and traversal.
3. Explain doubly linked list and its operations.
4. Explain circular linked list with a diagram.
5. Differentiate singly, doubly and circular linked lists.
6. Write an algorithm to insert and delete a node in a linked list.
7. Discuss applications, advantages and disadvantages of linked lists.

4. Stack

1. Define stack and explain the LIFO principle.
2. Explain stack operations: push, pop, peek and isEmpty.
3. Explain stack implementation using an array.
4. Explain stack implementation using a linked list.
5. What are stack overflow and underflow?
6. Explain applications of stack.
7. Explain how a stack is used for parentheses matching.

8. Convert an infix expression to postfix using a stack.

5. Queue

1. Define queue and explain the FIFO principle.
2. Explain enqueue and dequeue operations.
3. Explain queue implementation using an array.
4. Explain queue implementation using a linked list.
5. Differentiate stack and queue.
6. Explain applications of queues.
7. Explain queue overflow and underflow.

6. Circular Queue

1. What is a circular queue? Why is it needed?
2. Explain insertion and deletion in a circular queue.
3. Explain the conditions for empty and full circular queue.
4. Write an algorithm for enqueue and dequeue in a circular queue.
5. Compare simple queue and circular queue.
6. Explain applications of circular queues.

7. Recursion

1. Define recursion. Explain base case and recursive case.
2. Write a recursive algorithm for factorial.
3. Explain Fibonacci series using recursion.
4. Explain recursion with an example.
5. Compare recursion and iteration.
6. Explain applications and limitations of recursion.
7. Explain Tower of Hanoi using recursion.

8. Searching

1. Explain linear search with an example and algorithm.
2. Explain binary search with an example and algorithm.
3. Compare linear search and binary search.
4. What are the conditions required for binary search?
5. Analyze the time complexity of linear and binary search.
6. Write a program/algorithm to search an element in an array.

9. Sorting

1. Define sorting. Why is sorting required?
2. Explain Bubble Sort with an example and algorithm.

3. Explain Selection Sort with an example and algorithm.
4. Explain Insertion Sort with an example and algorithm.
5. Explain Merge Sort using divide-and-conquer.
6. Explain Quick Sort and the role of a pivot.
7. Compare Bubble, Selection and Insertion Sort.
8. Compare Merge Sort and Quick Sort.
9. Write the time complexities of major sorting algorithms.
10. Sort a given set of elements using a specified sorting algorithm.

10. Trees

1. Define tree and explain basic terminology: root, parent, child, leaf, degree, level and height.
2. Define binary tree. Explain its properties.
3. Explain different types of binary trees.
4. Explain representation of binary trees.
5. Define Binary Search Tree (BST) and explain its properties.
6. Explain insertion and deletion in a BST.
7. Explain searching in a BST.
8. Construct a BST for a given sequence of keys.

11. Tree Traversals

1. Explain preorder traversal with an example.
2. Explain inorder traversal with an example.
3. Explain postorder traversal with an example.
4. Explain level-order traversal with an example.
5. Differentiate preorder, inorder and postorder traversal.
6. Perform all traversals for a given binary tree.
7. Why does inorder traversal of a BST produce sorted order?

12. Balanced Search Trees – AVL

1. What is a balanced search tree? Why is balancing required?
2. Define AVL tree and balance factor.
3. Explain LL, RR, LR and RL rotations with diagrams.
4. Insert a sequence of keys into an AVL tree and show rotations.
5. Explain deletion in an AVL tree.
6. Compare BST and AVL tree.
7. Explain why AVL operations have $O(\log n)$ complexity.

13. Heap

1. Define heap. Explain the properties of a complete binary tree.

2. Differentiate min heap and max heap.
3. Explain insertion into a heap.
4. Explain deletion of the root from a heap.
5. Explain heapify operation.
6. Explain Heap Sort with an example.
7. Compare heap and binary search tree.
8. Explain applications of heaps and priority queues.

14. Hashing

1. Define hashing and hash function.
2. What is a collision? Why does it occur?
3. Explain collision resolution using separate chaining.
4. Explain open addressing.
5. Explain linear probing, quadratic probing and double hashing.
6. Compare chaining and open addressing.
7. Construct a hash table for a given set of keys.
8. Explain the average and worst-case complexity of hashing.

15. Graphs

1. Define graph. Explain vertices and edges.
2. Explain directed and undirected graphs.
3. Explain weighted and unweighted graphs.
4. Explain connected, disconnected, cyclic and acyclic graphs.
5. Explain adjacency matrix representation.
6. Explain adjacency list representation.
7. Compare adjacency matrix and adjacency list.
8. Explain graph traversal and its applications.

16. BFS and DFS

1. Explain Breadth-First Search (BFS) with an example.
2. Explain Depth-First Search (DFS) with an example.
3. Compare BFS and DFS.
4. Write the algorithm for BFS.
5. Write the algorithm for DFS.
6. Explain BFS implementation using a queue.
7. Explain DFS implementation using a stack or recursion.
8. Find the BFS/DFS traversal of a given graph.
9. State the time complexity of BFS and DFS.

17. Important Graph Algorithms

1. Explain Dijkstra's shortest path algorithm.
2. State the conditions/limitations of Dijkstra's algorithm.
3. Explain Prim's minimum spanning tree algorithm.
4. Explain Kruskal's minimum spanning tree algorithm.
5. Compare Prim's and Kruskal's algorithms.
6. Explain topological sorting and its applications.
7. Perform Dijkstra/Prim/Kruskal/topological sort on a given graph.

18. Exam-Focused 16-Mark Questions

1. Explain linked lists and all major operations with algorithms and diagrams.
2. Explain stack implementation and applications, including infix-to-postfix conversion.
3. Explain queue, circular queue and their operations with algorithms.
4. Explain searching and sorting algorithms with examples and complexity analysis.
5. Explain binary trees, BST operations and all tree traversal methods.
6. Explain AVL trees and all four rotations with suitable examples.
7. Explain heap operations and Heap Sort with an example.
8. Explain graph representations, BFS and DFS with algorithms and complexity.
9. Explain hashing and collision-resolution techniques with examples.
10. Explain major graph algorithms: Dijkstra, Prim, Kruskal and topological sorting.

Quick Exam Revision

Topic	Must-prepare areas
Arrays	Operations, insertion/deletion, complexity
Linked Lists	Singly/doubly/circular, operations
Stack	Operations, implementation, applications, expressions
Queue	FIFO, operations, circular queue
Searching	Linear vs binary search + complexity
Sorting	Bubble, selection, insertion, merge, quick
Trees	Terminology, binary tree, BST
Traversal	Preorder, inorder, postorder, level order
AVL	Balance factor + four rotations
Heap	Min/max heap, heapify, heap sort
Hashing	Hash function + collision resolution
Graphs	Representations, BFS, DFS
Graph Algorithms	Dijkstra, Prim, Kruskal, topological sort